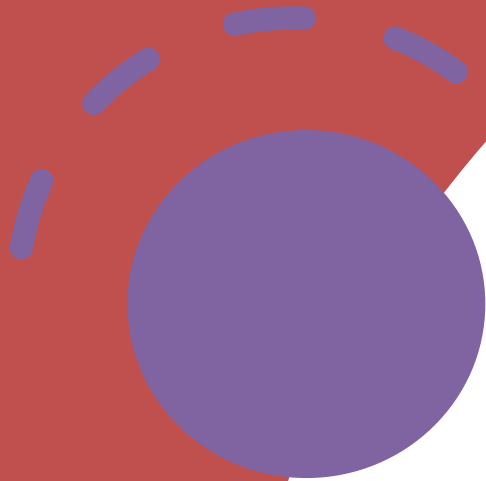




Análisis Básico de algoritmos

Estructuras de Datos y Algoritmos I

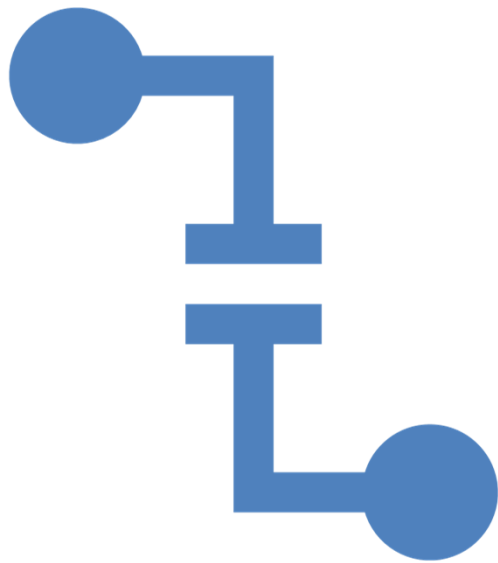
Profesora: Elba Karen Sáenz García

FI UNAM

Conceptos Importantes en computación

Problema

Algoritmo



Problema

Conjunto de Instancias al cual se le asocia un conjunto de soluciones



Tipos de problemas

Problemas de decisión

- La respuesta es si o no

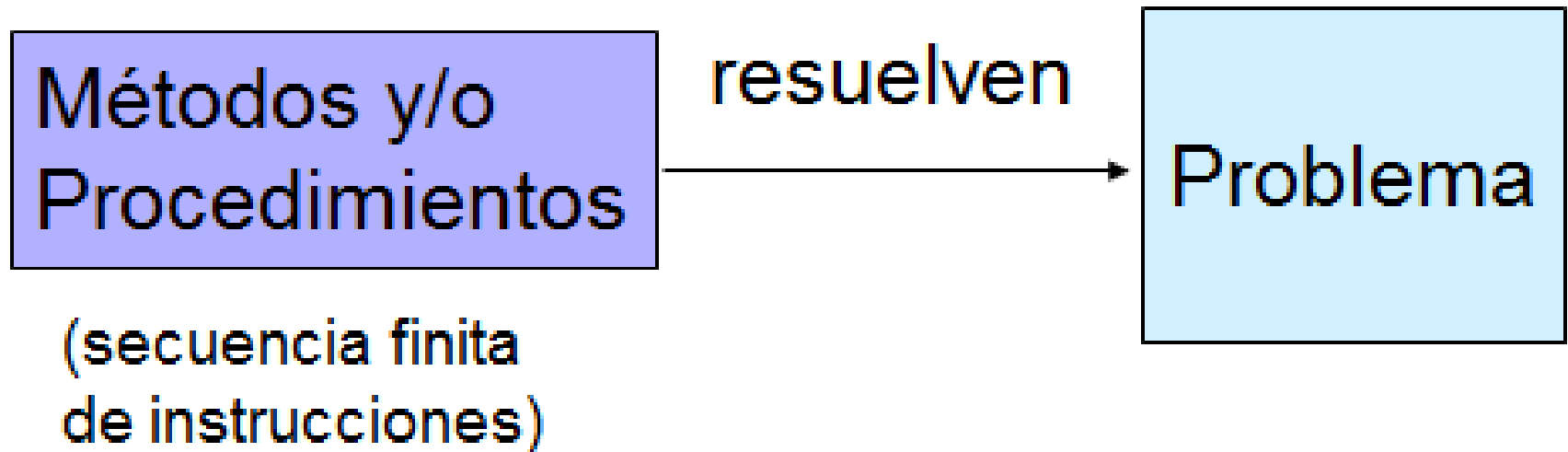
Problemas de optimización

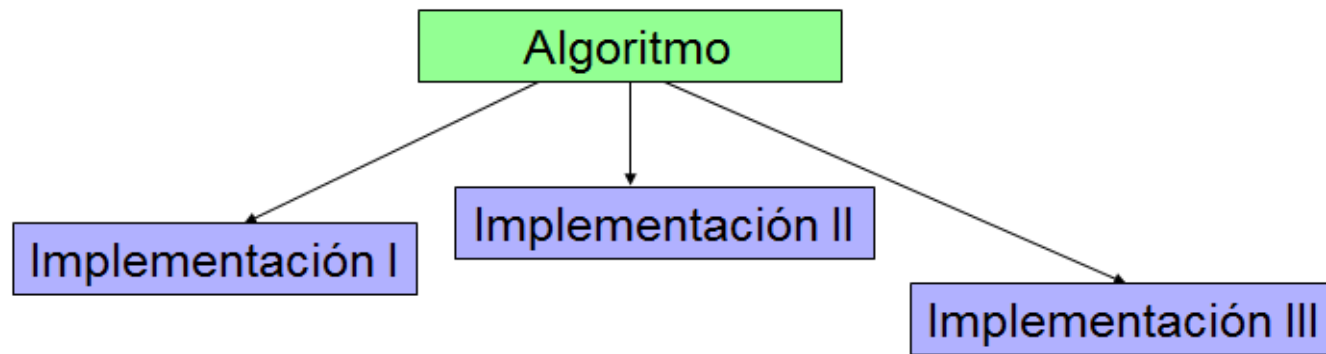
- Respuesta a la pregunta ¿Cómo se obtiene el mejor valor posible?

Algoritmo

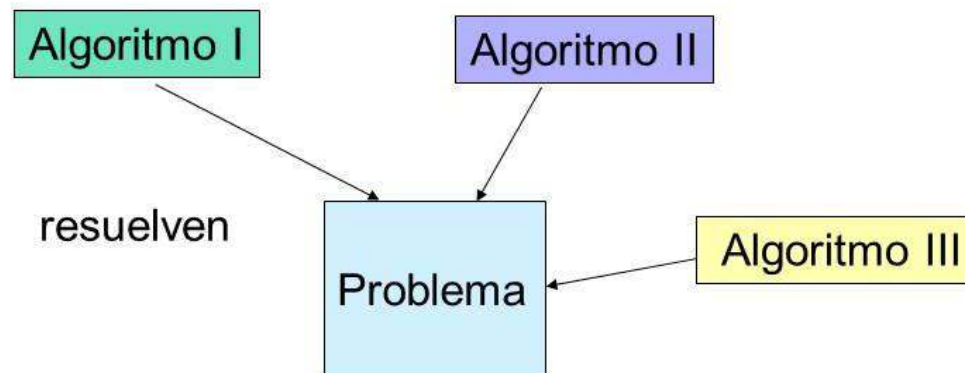
- Es un método de solución para resolver una instancia específica de un determinado problema.

Se buscan:





Algoritmo



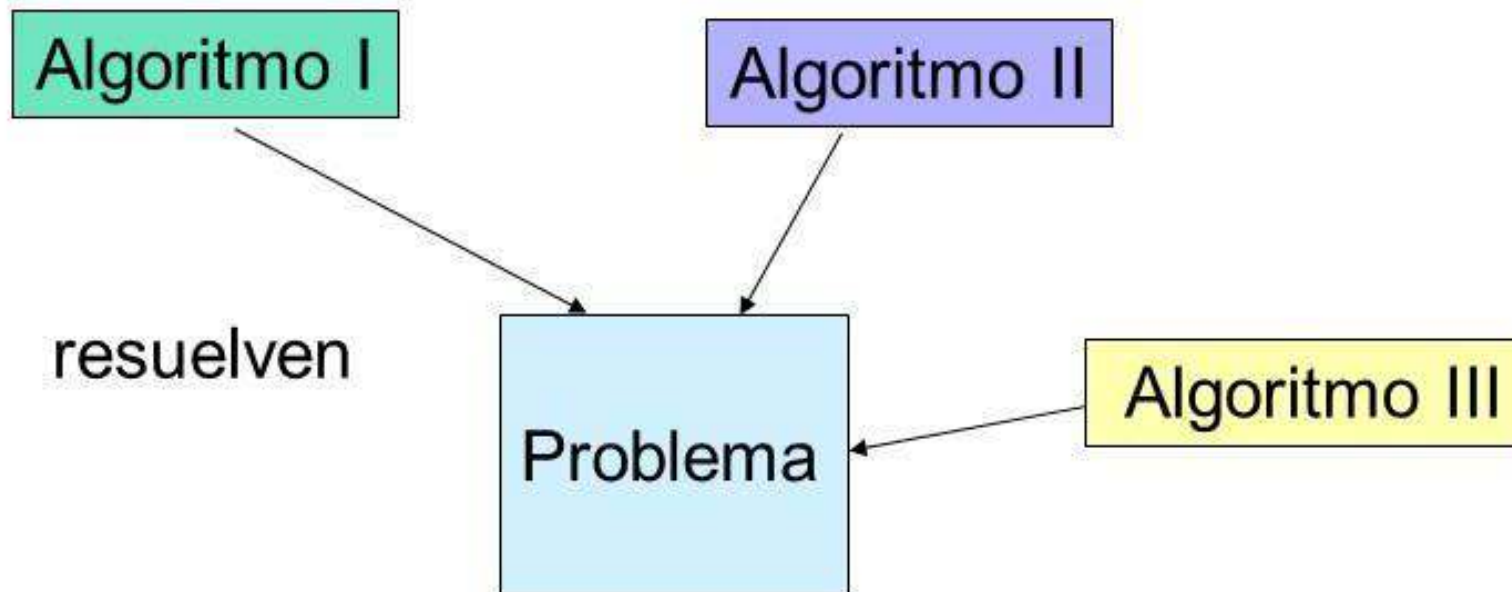
Elementos primordiales de Algoritmos

El conjunto E
de entradas

Conjunto S
de salidas

Análisis de
algoritmos

Analizar dos o más algoritmos que resuelven el mismo problema para determinar cuál de ellos requiere menos recursos.



Análisis de algoritmos

Enfoques

- Benchmarking
- Análisis Matemático de Algoritmos
- Análisis Asintótico de Algoritmos

Análisis de algoritmos

Bechmarking

- Consiste en implementar los algoritmos en cuestión, ejecutarlos y determinar cuál de ellos requiere menos recursos.
- Esta solución parece ser la más sencilla e intuitiva, sin embargo tiene varios inconvenientes

Análisis de
algoritmos.

Análisis
Matemático

- Consiste en tratar de asociar con cada algoritmo una función matemática exacta que mida su “eficiencia” dependiendo sólo de los siguientes factores:
 - Las características estructurales del algoritmo.
 - El tamaño del conjunto de datos de entrada

Análisis asintótico de algoritmos

- Técnica derivada del análisis matemático de algoritmos basada en dos conceptos fundamentales:
 - la caracterización de datos de entrada y
 - la complejidad asintótica.

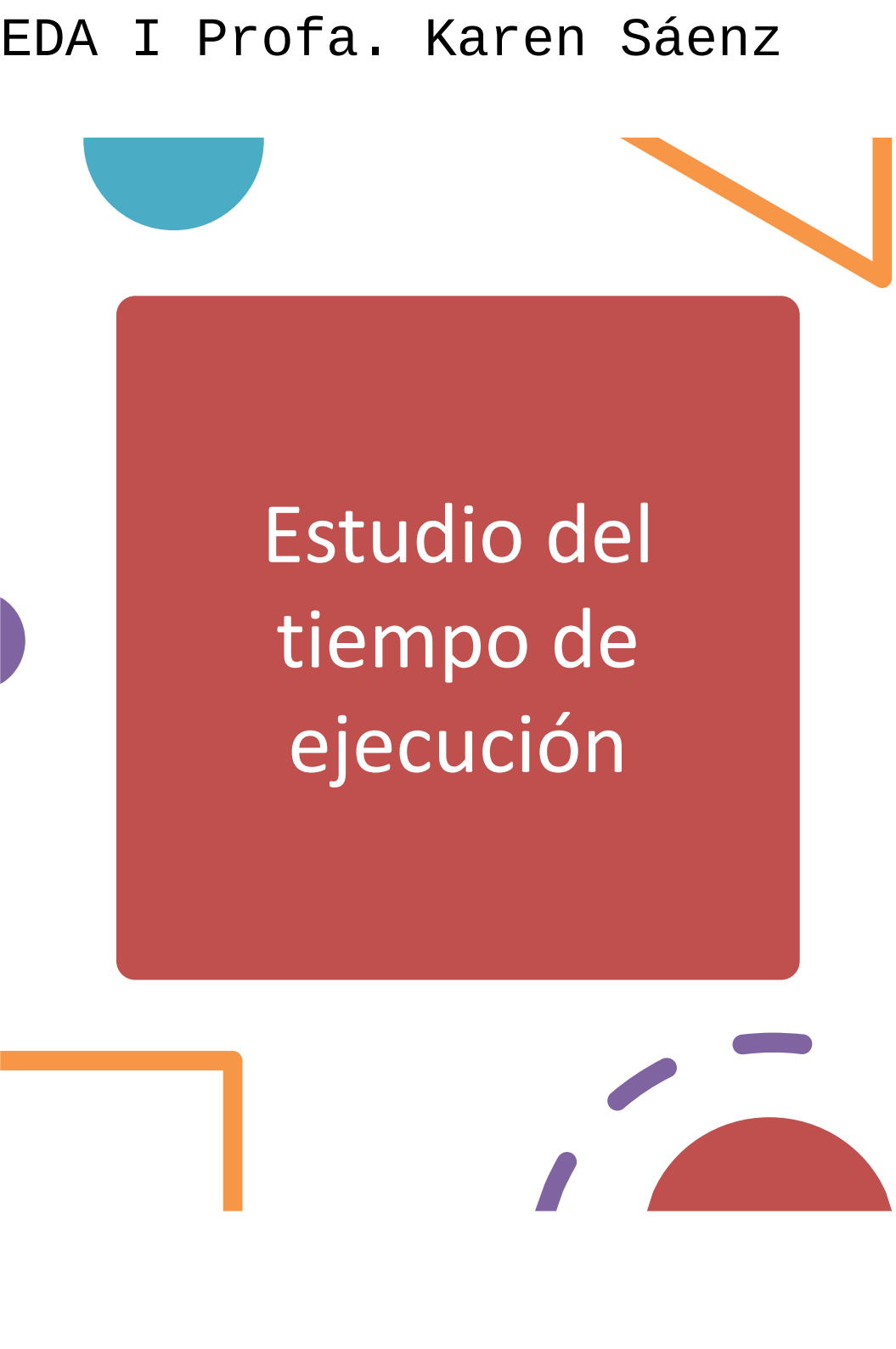
Eficiencia de los Algoritmos

- Se llama eficiencia de un algoritmo a la relación entre los **recursos** que consume y los resultados que obtiene.
- Los aspectos a tomar en cuenta para estudiar la *eficiencia* de un algoritmo son:
 - El tiempo que se emplea en resolver el problema
 - y la cantidad de recursos de memoria que ocupa.

Factores que influyen en el consumo de recursos

- Máquina en la que se ejecuta
- Lenguaje de programación
- Tipos de datos utilizados
- Compilador y opciones de compilación
- Usuarios que trabajan en el sistema





Estudio del tiempo de ejecución

- Por factores externos no se puede predecir el tiempo de ejecución.
- Su estudio debe ser independiente de esos factores.



Análisis Matemático

Análisis de Algoritmos
Estructura de Datos y Algoritmos I



Tiempo de ejecución de un algoritmo

- 
- Se le llama *tiempo de ejecución*, no al tiempo físico, sino al *número de operaciones elementales OE* que se llevan a cabo en el algoritmo.
 - Se representa como una función $T(n)$

Operaciones Elementales

El estudio se suele hacer con el conteo de instrucciones (operaciones elementales)

- Operación aritmética.
- Asignación a una variable.
- Llamada a una función.
- Retorno de una función.
- Comparaciones lógicas (con salto).
- Acceso a una estructura (arreglo, matriz, lista ligada...).

Tiempo de ejecución de un algoritmo

- El tiempo $T(n)$ esta en función de n que representa el **tamaño o talla del problema** o instancia.

Talla o tamaño del problema

- Es cualquier parámetro en función del cual se pueda expresar $T(n)$:
 - N° de datos de entrada
 - N° de datos de salida
 - Valor de las variables numéricas
 - Una función de los anteriores
- Suele guardar relación con el volumen de los datos a tratar, y por ello se le suele llamar “tamaño” del problema.

Talla o tamaño del problema

Ejemplos

- 1.- Algoritmo de ordenación: la talla del problema (n) será el número de elementos a ordenar.
- 2.- Algoritmo de búsqueda de un elemento en una secuencia: la talla del problema será el número de elementos de la secuencia.
- 3.- Algoritmo del cálculo del factorial de un número: la talla será el valor del número.

Tipos de estudio de $T(n)$

- **Tiempo en el caso mas favorable (mejor caso):** El tiempo de ejecución con la entrada que produce tiempo mínimo
- **Tiempo en el caso mas desfavorable(peor caso):** El tiempo de ejecución con la entrada que produce tiempo máximo
- **Tiempo promedio:** Promedio de cada una de las posibles entradas por la probabilidad de que estas aparezcan.

Ejemplo 1

- Talla del problema: n .
- Instancias:



Caso mejor



Caso genérico



Caso peor

- Búsqueda secuencial de un elemento x en un arreglo A .

Cálculo $T(n)$

- En cuanto al tiempo, se estudia el número de operaciones elementales *en el peor caso, en el mejor caso o en el caso promedio.*
- *Si se estudia el peor caso tendremos una idea de cuanto será lo más que podrá tardar el algoritmo.*
- Cuando es de interés, se estudia el caso promedio.

Ejemplo1

Int main(){		Mejor caso
int a=2,b=3,c=0;	(3)	$T(n)=3+2+1+1+1+1=9$
c=a+b;	(2)	
if(c>0)	(1)	Peor caso
printf(“%d es mayor a 0”, c);	(1)	$T(n)=3+2+1+\mathbf{2}+1+1=10$
else{		
c=c*(-1);	(2)	
}		
printf(“%d”, c);	(1)	
return 0;	(1)	
}		

Ejemplo 2

- Análisis de operaciones elementales en un algoritmo que busca un número dado dentro de un arreglo ordenado.

Ejemplo 2

Tamaño del Problema : Número de elemento del arreglo a en n.

2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	----	----

```
int buscar(int n, VectorInt a, int c) {  
    int r;  
    int j = 0;                                // 1  
    while (a[j]<c && j<n-1) {                 // 2  
        j++;                                  // 3  
    }                                          // 4  
    if (a[j]==c ) {                           // 5  
        r = j;                                // 6  
    } else {                                  // 7  
        r = -1;                               // 8  
    }                                          // 9  
    return r;  
}
```

Mejor Caso

```

int buscar(int n, VectorInt a, int c) {
    int r;
    int j = 0;
    while (a[j]<c && j<n-1) {
        j++;
    }
    if (a[j]==c) {
        r = j;
    } else {
        r = -1;
    }
    return r;
}

```

// 1 1 OE
 // 2 5 OE
 // 3 2 OE
 // 4
 // 5 2 OE
 // 6 1 OE
 // 7
 // 8 1 OE
 // 9

$$T(n) = 1+2+2+1=6$$

Cuando el número buscado se encuentra inmediatamente en el primer elemento del arreglo

Peor Caso

$$T(n) = 1 + \left(\sum_{j=0}^{n-2} (5 + 2) \right) + 5 + 2 + 1 = 9 + 7(n-1) = 7n + 2$$

```

int buscar(int n, VectorInt a, int c) {
    int r;
    int j = 0; // 1    1 OE
    while (a[j] < c && j < n-1) { // 2    5 OE
        j++; // 3    2 OE
    } // 4
    if (a[j] == c) { // 5    2 OE
        r = j; // 6    1 OE
    } else { // 7
        r = -1; // 8    1 OE
    } // 9
    return r;
}

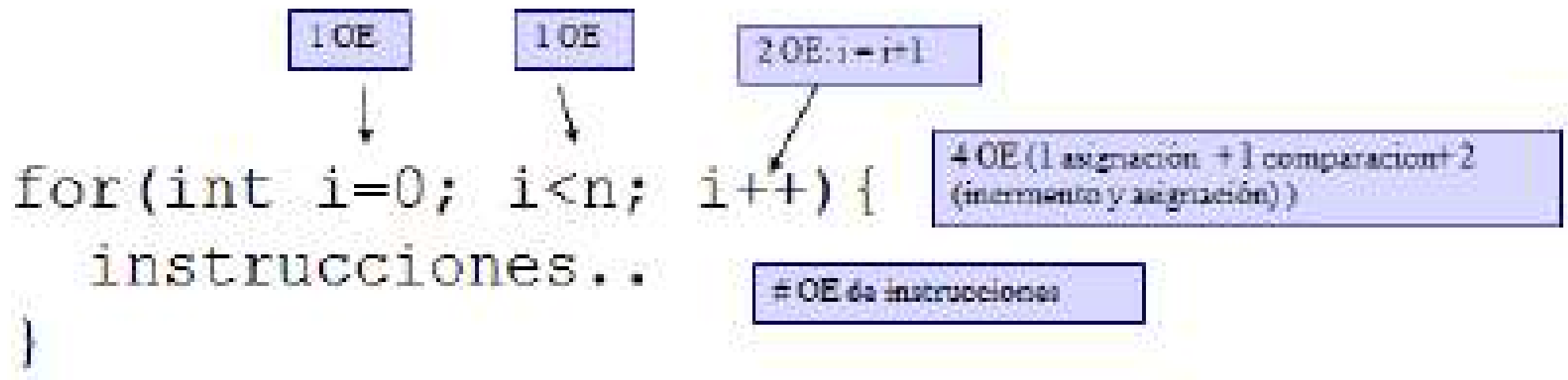
```

Cuando se recorre todo el arreglo y no se encuentra al elemento buscado.

Caso Promedio

- Para este algoritmo, todos tienen la misma probabilidad de suceder
- Promedio de cada una de las posibles entradas por la probabilidad de que estas aparezcan.

Otros ejemplos



El tiempo total en el peor caso es:

$$1 + 1 + \sum_{i=0}^{n-1} (\text{\#OE de instrucciones} + 2 + 1)$$

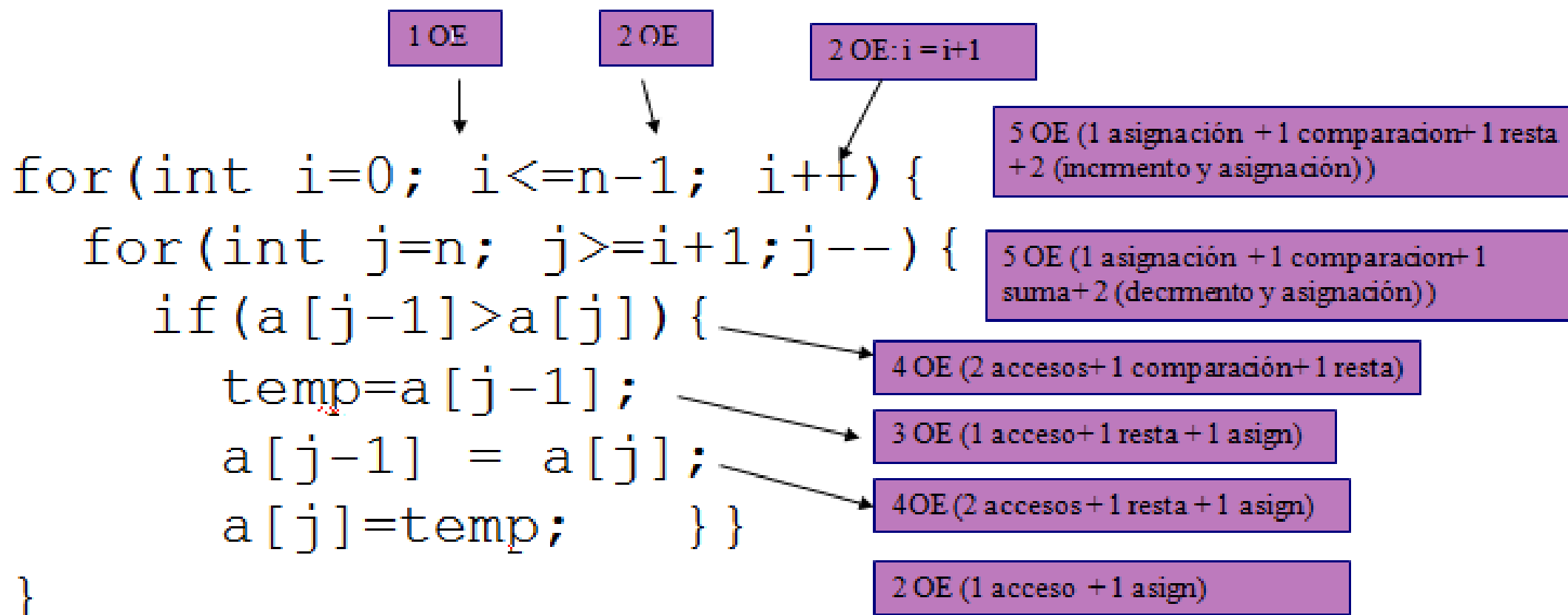
```

int i=0      [1 OE]
while( i<n ) { [1 OE]
    instrucciones... [#OE de instrucciones]
    i=i+1;    [2 OE: i - i+1]
}

```

El tiempo total en el peor caso es:

$$1 + 1 + \sum_{i=0}^{n-1} (\text{\#OE de instrucciones} + 2 + 1)$$



$$T_2(n) = 1 + 2 + \sum_{j=i+1}^n (13 + 2 + 2) =$$

$$T(n) = 1 + 2 + \sum_{i=1}^{n-1} (\#forinterno + 2 + 2)$$

$$T2(n) = 1 + 2 + \sum_{j=i+1}^n (13 + 2 + 2) =$$

$$T2(n) = 3 + (n - i - 1 + 1)17 = 3 + (n - i)17 = 3 + 17n - i17$$

$$T(n) = 3 + \sum_{i=1}^n (3 + 17n - 17i + 4) = 3 + \sum_{i=1}^n (7 + 17n - 17i)$$

$$= 3 + 7n + 17n^2 - 17 \left(\frac{n(n+1)}{2} \right) = 3 - \frac{3}{2}n + \frac{17}{2}n^2$$

$$T(n) = 3 - \frac{3}{2}n + \frac{17}{2}n^2$$

```
if( a > b ){
```

1OE

```
    a = 2*b +c;
```

3OE (1 mult + 1 suma + 1 asign)

```
    c = a/2;
```

2OE (1 div + 1 asign)

```
    b = a+c;
```

2OE (1 div + 1 asign)

```
    return a*b;
```

2OE

```
}
```

$T(cuerpo) = 9$

Cálculo de $t(n)$ o $T(n)$

Cálculo de $T(n)$:

Secuencia de operaciones elementales:

- $T(n) = k$

Composición secuencial: $\{s1, \dots, sr\}$

- $T(n) = T1(n) + \dots + Tr(n)$
- Donde $Ti(n)$ es el tiempo de ejecución de Si

Instrucción condicional: *if(g) s1 else s2;*

- Caso peor: $T(n) = Tg(n) + \max(T1(n), T2(n))$
- Caso mejor: $T(n) = Tg(n) + \min(T1(n), T2(n))$
- Caso medio: $T(n) = Tg(n) + \text{med}(T1(n), T2(n))$